

СОГЛАСОВАНО

Представитель Заказчика

«WebStyle»

_____ А. Б. Вгдшник

«__» _____ 2026 г.

Техническое задание и аналитическое описание
ИНТЕГРАЦИИ CRM «WEBSTYLE» С MADETASK
(версия 1.0 от 27.04.2026)

Системный аналитик

А. М. Изимова

Санкт-Петербург
2026

РЕЗЮМЕ ПРОЕКТА

Компания WebStyle разрабатывает сайты под ключ и привлекает внутренних сотрудников и внешних подрядчиков. Внутренняя команда работает в CRM, а внешние дизайнеры и копирайтеры получают задания через мессенджеры. Из-за этого отсутствует единый источник данных по задачам, приемке и выплатам.

Данный проект направлен на интеграцию CRM WebStyle с облачным сервисом MadeTask, чтобы сохранить работу пользователей в CRM, но использовать MadeTask для учета внешних исполнителей, подтверждения выполненных задач и автоматизации выплат.

Целевое решение:

- CRM остается основным интерфейсом для менеджеров, бухгалтерии и внешних подрядчиков.
- MadeTask используется как внешняя система учета исполнителей и платежной инфраструктуры.
- Обмен выполняется через API: создание задач, изменение статуса выполнения, инициирование выплаты и получение статуса операции.

Таблица 1 – Характеристики системы

Параметр	Описание
Бизнес-цель	Снизить ручной труд менеджеров и бухгалтерии, исключить потерю задач и обеспечить прозрачность выплат по проектам.
Ключевой результат	Все задачи и статусы видны в CRM; выплаты подрядчикам запускаются автоматически после приемки работы менеджером.
Основные системы	CRM WebStyle, MadeTask API, банк/платежная инфраструктура через MadeTask, уведомления CRM.
Границы проекта	Интеграция с MadeTask и доработка CRM. Разработка интерфейса MadeTask не выполняется; подрядчики продолжают работать в CRM.
Критичные условия	Исполнитель должен самостоятельно зарегистрироваться в MadeTask, пройти верификацию и привязать счет/платежный инструмент до первой выплаты.

Данное техническое задание и аналитическое описание интеграции систем содержит 13 таблиц, 7 приложений.

СОДЕРЖАНИЕ

Термины и определения.....	5
1 АНАЛИЗ ТЕКУЩИХ ПРОЦЕССОВ	6
1.1 Цель системы	6
1.2 Основные элементы системы.....	6
1.3 Взаимосвязи между элементами.....	7
1.4 Основные свойства элементов системы	8
1.5 Основные свойства системы в целом	9
Выводы	10
2 ПРОЕКТИРОВАНИЕ БУДУЩЕГО РЕШЕНИЯ	12
2.1 Цель и принципы целевой системы (ТО ВЕ).....	12
2.2 Общая схема целевого решения	13
2.3 Роль пользователей в целевой системе	14
2.4 Диаграмма вариантов использования.....	15
2.5 Описание ключевых сценариев (ТО ВЕ).....	16
2.6 Изменение бизнес-сущностей.....	17
Выводы	18
3 ИНТЕГРАЦИЯ СИСТЕМ	19
3.1 Назначение интеграционного блока.....	19
3.2 Участники интеграции	19
3.3 Тип интеграции.....	20
3.4 Основные интеграционные сценарии.....	21
3.5 Требования к интеграционному обмену	21
Выводы	22
4 РАБОТА С API MADETASK	23
4.1 Назначение API MadeTask в проекте	23
4.2 Общие принципы взаимодействия	23
4.3 Базовые технические условия обмена.....	24
4.4 Методы API и данные	24
4.5 Примеры запросов	26
4.6 Ограничения и требования к безопасности	30
Выводы	31
5 ДОКУМЕНТИРОВАНИЕ	32
5.1 Назначение раздела	32

5.2 Границы решения	32
5.3 Функциональные требования (FR)	32
5.4 Нефункциональные требования (NFR)	34
5.5 Список задач разработки (Backlog)	35
5.6 План реализации	35
5.7 Критерии приемки	36
Выводы	36
Заключение	37
ПРИЛОЖЕНИЯ	38
Приложение А – Процесс AS IS (WebStyle)	38
Приложение Б – Процесс TO BE (WebStyle)	39
Приложение В – Use-Case диаграмма	40
Приложение Г – Бизнес-сущности	41
Приложение Д – UML Sequence: создание задачи и проверка исполнителя	44
Приложение Е – UML Sequence: Обработка ошибок интеграции	47
Приложение Ж – UML Sequence: Сценарий отсутствия счета у исполнителя	50

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

API (Application Programming Interface)	– программный интерфейс, через который CRM WebStyle взаимодействует с MadeTask
	–
CRM (Customer Relationship Management)	– информационная система компании WebStyle, используемая для управления проектами, задачами, пользователями и взаимодействием с исполнителями. Является основной системой (master system) в рамках интеграции
	–
MadeTask	– облачный сервис, предоставляющий API для учета исполнителей, управления задачами и автоматизации выплат

1 АНАЛИЗ ТЕКУЩИХ ПРОЦЕССОВ

1.1 Цель системы

Текущая система компании «WebStyle» предназначена для управления процессом разработки сайтов, включая:

- постановку и контроль выполнения задач;
- взаимодействие с внутренними сотрудниками и внешними подрядчиками;
- учет выполненных работ и последующих выплат.

Основная цель системы — обеспечить выполнение проектов в срок с контролем качества и затрат.

Фактически система существует для:

- координации работы команды;
- распределения задач;
- обеспечения взаимодействия между участниками проекта;
- проведения расчетов с исполнителями.

Однако на текущий момент система выполняет эту функцию частично и не централизованно, что приводит к операционным потерям.

1.2 Основные элементы системы

Текущая система представляет собой распределенную (несвязанную) систему, состоящую из нескольких независимых компонентов.

Основной уровень — бизнес-система управления проектами

CRM-система (ядро системы)

- Хранение проектов
- Управление задачами (частично)
- Работа внутренних сотрудников

Внешние вспомогательные системы

Коммуникационные каналы - Мессенджеры (Telegram, WhatsApp и др.)

- Передача ТЗ дизайнерам
- Обсуждение правок

- Передача результатов

Файловые хранилища

- Обмен макетами и материалами (Google Drive, локальные хранилища)

Финансовый контур

Excel / таблицы учета

- Ведение реестра выплат
- Хранение реквизитов исполнителей

Банковские системы / СБП

- Проведение платежей вручную

Участники системы

Внутренние пользователи

- Менеджеры проектов
- Разработчики
- Бухгалтерия

Внешние пользователи

- Дизайнеры
- Копирайтеры
- Другие подрядчики

1.3 Взаимосвязи между элементами

Диаграмма AS IS полного процесса (постановка, приемка и ручная оплата задачи внешнему дизайнеру) приведена в приложении А. Взаимодействие элементов системы носит фрагментированный и несинхронизированный характер.

Основные потоки взаимодействия:

Постановка задач

- Менеджер создает задачу в CRM (или не создает)
- Параллельно отправляет ТЗ дизайнеру через мессенджер

Выполнение задачи

- Дизайнер выполняет работу

- Результат отправляется в чат

Проверка

- Менеджер проверяет результат вручную
- Обратная связь — через мессенджер

Фиксация результата

- Может быть зафиксирована в CRM, а может отсутствовать

Формирование оплаты

- Бухгалтер получает информацию от менеджера
- Проверяет данные в Excel
- Проведение выплаты
- Ручной перевод через банк/СБП

Заккрытие задачи

- Фактически не синхронизировано с оплатой

Таким образом, ключевыми проблемами связей являются:

- отсутствует единый источник истины (Single Source of Truth);
- отсутствует автоматическая синхронизация между задачами и оплатами;
- коммуникации не связаны с системой учета.

1.4 Основные свойства элементов системы

CRM-система

- Централизованное хранение проектов
- Ограниченная роль в управлении внешними исполнителями
- Не интегрирована с финансовыми процессами

Мессенджеры

- Высокая скорость коммуникации
- Отсутствие структурированности
- Потеря данных и истории задач

Excel / учетные таблицы

- Гибкость хранения данных

- Высокая зависимость от ручного ввода
- Ошибки при копировании и сверке

Банковские системы

- Надежность проведения платежей
- Полное отсутствие автоматизации
- Нет связи с задачами и проектами

Внешние подрядчики

- Не интегрированы в CRM
- Работают вне системы
- Нет прозрачного учета их активности

Менеджеры

- Центральная роль в процессе
- Высокая операционная нагрузка
- Ручное управление всеми этапами

Бухгалтерия

- Ручная обработка выплат
- Проверка данных из разных источников
- Высокий риск ошибок

1.5 Основные свойства системы в целом

1. Разрозненность (децентрализация)

Система состоит из несвязанных компонентов, между которыми отсутствует единый контур управления.

2. Низкая прозрачность

Нет полной картины затрат по проекту

Невозможно быстро определить статус оплаты

3. Высокая зависимость от человека

Ключевые процессы выполняются вручную

Ошибки зависят от человеческого фактора

4. Низкая масштабируемость

Увеличение числа подрядчиков приводит к экспоненциальному росту нагрузки

Система не выдерживает рост без ухудшения качества

5. Отсутствие трассируемости

Нет связи: задача → выполнение → оплата

Сложно провести аудит

6. Низкая управляемость процессов

Нет автоматических триггеров

Нет контроля SLA

7. Открытость системы (неуправляемая)

Используются внешние каналы (мессенджеры, таблицы)

Нет контроля и интеграции

Таким образом, основные проблемы действующего бизнес-процесса представлены в Таблице 2.

Таблица 2 – Основные проблемы и их причины

Проблема	Причина	Последствие
Потеря задач и правок	ТЗ и результаты находятся в чатах без единого статуса и версии.	Срывы сроков, повторные вопросы, конфликт по объему работ.
Ручные выплаты	Реквизиты и суммы сверяются в Excel, платежи создаются вручную.	Ошибки в реквизитах, высокая нагрузка на бухгалтерию, задержки выплат.
Нет прозрачности по проекту	Платежи не связаны с задачами и проектами в CRM в момент оплаты.	Менеджер и руководитель видят расходы с задержкой.
Сложная работа с внешними исполнителями	Внешние подрядчики не включены в единый контур задач.	Невозможно быстро оценить загрузку, историю задач и сумму выплат.
Риск оплаты без приемки	Нет системной связи между статусом «Принято» и платежом.	Финансовые и операционные риски.

Выводы

Текущая система компании «WebStyle» представляет собой частично цифровизированную, но не интегрированную среду, в которой процессы управления задачами и оплаты разорваны; отсутствует единый контур контроля; ключевые операции выполняются вручную.

Это приводит к операционным и финансовым издержкам, а также большой нагрузке на сотрудников.

Данная ситуация обосновывает необходимость внедрения интеграционного решения с использованием MadeTask API для централизации процессов и автоматизации выплат.

2 ПРОЕКТИРОВАНИЕ БУДУЩЕГО РЕШЕНИЯ

2.1 Цель и принципы целевой системы (TO BE)

Целевая система направлена на устранение выявленных проблем текущего процесса за счет интеграции CRM компании «WebStyle» с сервисом MadeTask.

Основной целью внедрения является создание единого управляемого контура, в котором:

- все задачи (внутренние и внешние) ведутся в CRM;
- статусы задач синхронизированы с системой выплат;
- оплата выполняется автоматически на основании факта приемки работы;
- исключаются ручные операции и дублирование данных.

Ключевые принципы решения:

1. Единая точка управления (Single Source of Truth)
 - CRM является единственной системой для работы пользователей.
 - Прозрачная связка “задача → выполнение → оплата”
 - Каждая задача имеет прямую связь с финансовой операцией.
2. Автоматизация выплат
 - Оплата инициируется системой без участия бухгалтерии.
 - Изоляция внешнего сервиса
 - Внешние исполнители не взаимодействуют напрямую с MadeTask.
3. Событийная модель взаимодействия
 - Изменение статусов задач автоматически инициирует действия в системе.

В рамках проектируемого решения принимаются следующие ограничения и допущения:

- сервис MadeTask используется как внешний облачный компонент;
- CRM не хранит платежные реквизиты исполнителей — они находятся в MadeTask;

- регистрация исполнителя в MadeTask осуществляется самостоятельно по предоставленной ссылке;
- одна задача соответствует одной выплате;
- изменения статусов в CRM являются источником событий для интеграции;
- гарантируется сетевое взаимодействие между CRM и MadeTask;
- операции API могут выполняться асинхронно.

2.2 Общая схема целевого решения

Целевая архитектура представлена в Приложении Б —схема ТО ВЕ.

Система включает следующие компоненты:

1. CRM WebStyle (ядро системы)

- управление задачами;
- управление пользователями;
- хранение статусов;
- интерфейс для всех ролей;
- инициирование интеграционных операций.

2. MadeTask (внешний сервис)

- учет исполнителей;
- управление задачами для оплаты;
- проведение выплат;
- контроль платежных операций.

3. Интеграционный слой (API)

- создание задач;
- передача статусов;
- инициирование выплат;
- обработка ошибок.

Процесс включает:

- постановку задачи;
- проверку исполнителя;

- выполнение работы;
- приемку;
- автоматическую выплату.

Ключевые улучшения относительно AS IS:

- исключены мессенджеры как основной канал;
- исключены Excel-таблицы;
- устранены ручные платежи;
- обеспечена сквозная трассируемость процесса.

2.3 Роль пользователей в целевой системе

Внутренние пользователи

Менеджер проекта

- создает задачи;
- назначает исполнителей;
- контролирует выполнение;
- принимает результат;
- инициирует (косвенно) выплату через приемку задачи.

Бухгалтерия

- контролирует результаты выплат;
- анализирует финансовые отчеты;
- не участвует в ручных переводах.

Администратор

- управляет ролями и доступами;
- контролирует настройки интеграции;
- обеспечивает корректность работы API.

Внешние пользователи

Внешний исполнитель (дизайнер, копирайтер)

- получает задачи в CRM;
- выполняет работу;
- отмечает задачу как выполненную;

- регистрируется в MadeTask для получения выплат.

Важно отметить, что исполнитель не взаимодействует напрямую с MadeTask, кроме этапа регистрации. Процесс регистрации исполнителя является условно-обязательным и выполняется только один раз для каждого исполнителя. Повторная регистрация не требуется.

Аналогично часть процессов, связанных с банковскими расчетами, не является самостоятельным целевым бизнес-процессом в рамках данной работы, а рассматривается как вспомогательный процесс.

2.4 Диаграмма вариантов использования

Use-Case диаграмма представлена в Приложении В.

В рамках системы выделены следующие ключевые сценарии:

UC-1: Регистрация исполнителя в MadeTask

Цель: подготовка исполнителя к получению выплат.

UC-2: Создание задачи

Цель: формирование задачи и ее регистрация в MadeTask.

UC-3: Выполнение задачи

Цель: выполнение и передача результата.

UC-4: Приемка задачи

Цель: контроль качества и подтверждение выполнения.

UC-5: Автоматическая выплата

Цель: проведение оплаты исполнителю.

В рамках процесса предусмотрены следующие исключительные сценарии:

- исполнитель не зарегистрирован в MadeTask;
- ошибка создания задачи в MadeTask;
- возврат задачи на доработку;
- ошибка выполнения выплаты;
- отсутствие ответа от API MadeTask.

Для каждого сценария предусмотрен механизм:

- повторной попытки;
- уведомления пользователя;
- фиксации ошибки в системе.

2.5 Описание ключевых сценариев (ТО BE)

2.5.1 Сценарий 1 — Создание задачи

Менеджер создает задачу в CRM.

Указывает:

исполнителя;

описание;

стоимость.

Система проверяет статус исполнителя.

Условие: исполнитель зарегистрирован в MadeTask?

Нет → запускается процесс регистрации.

Да → продолжается создание задачи.

CRM создает задачу в MadeTask через API.

Получает уникальный идентификатор.

Сохраняет связь между системами.

Результат:

задача существует в обеих системах;

статус: «в работе».

2.5.2 Сценарий 2 — Выполнение задачи

Исполнитель выполняет задачу.

Отмечает статус «Готово» в CRM.

CRM синхронизирует статус с MadeTask.

Результат:

задача переходит в статус «на проверке».

2.5.3 Сценарий 3 — Приемка задачи

Менеджер проверяет результат.

Условие: работа соответствует требованиям?

Нет:

задача возвращается в статус «на доработке»;
исполнитель повторно выполняет работу.

Да:

задача принимается;
CRM отправляет подтверждение в MadeTask.

Результат: статус задачи: «выполнена».

2.5.4 Сценарий 4 — Автоматическая выплата

MadeTask инициирует выплату.

Выполняется проверка:

наличие счета;
корректность данных.

Альтернативный сценарий: ошибка выплаты
фиксируется ошибка;

CRM получает уведомление;
требуется корректировка данных.

Основной сценарий:
выплата выполняется;
статус: «выплачено».

CRM получает результат операции.

Платеж фиксируется в системе.

Результат: задача закрыта; оплата выполнена.

2.6 Изменение бизнес-сущностей

В целевой системе ключевыми бизнес-сущностями являются задача CRM, задача MadeTask, исполнитель, результат работы, платеж и интеграционный запрос.

В системе устанавливаются следующие ключевые связи:

задача CRM ↔ задача MadeTask (1:1);

задача ↔ исполнитель (N:1);

задача ↔ платеж (1:1);

исполнитель ↔ платеж (1:N);

задача ↔ результат работы (1:1).

Данные связи обеспечивают сквозную трассируемость: задача → выполнение → оплата

Для основных сущностей определены допустимые статусы и переходы и представлены в расширенном формате в Приложении Г. Взаимодействие между CRM и MadeTask реализуется по событийной модели, при которой изменения статусов бизнес-сущностей инициируют вызовы API. Статусная модель обеспечивает прозрачность процесса и исключает неконсистентные состояния, например оплату задачи без приемки работы.

Выводы

В результате проектирования целевого решения сформирована модель бизнес-процесса, обеспечивающая сквозное управление задачами и автоматизацию выплат внешним исполнителям. Целевая система устраняет ключевые недостатки текущего процесса за счет:

- централизации управления задачами в CRM;
- синхронизации статусов между системами;
- автоматизации финансовых операций;
- внедрения событийной модели взаимодействия;
- обеспечения полной трассируемости жизненного цикла задачи.

Определены роли пользователей, ключевые сценарии взаимодействия, статусные модели бизнес-сущностей и контрольные точки процесса.

Таким образом, сформирована логическая основа для реализации интеграции между CRM WebStyle и сервисом MadeTask.

На основании описанных бизнес-сценариев и потоков данных в следующем разделе будет детализировано взаимодействие систем на уровне интеграции, включая последовательности вызовов API и обмен сообщениями между CRM и MadeTask.

3 ИНТЕГРАЦИЯ СИСТЕМ

3.1 Назначение интеграционного блока

Интеграция CRM WebStyle с MadeTask необходима для автоматизации полного цикла работы с внешними исполнителями: от создания задачи до фиксации выплаты. В целевом решении CRM остается основной системой для пользователей, а MadeTask используется как внешний облачный сервис для регистрации задач, подтверждения выполнения и проведения выплат.

Интеграция выполняется через API-взаимодействие между системами. Такой подход соответствует логике API-интеграции, при которой системы обмениваются данными через стандартные интерфейсы клиент–сервер. В учебном материале по сетевому взаимодействию указано, что аналитику важно описывать не только бизнес-логику, но и то, как системы будут взаимодействовать: через какой механизм, в каком формате и с какими ограничениями.

3.2 Участники интеграции

Менеджер проекта: Создает задачу, проверяет результат, принимает работу

Внешний исполнитель: Получает задачу, выполняет работу, отмечает готовность

CRM WebStyle: Основная система, хранит задачи, статусы, пользователей и платежные операции

Интеграционный модуль CRM: Выполняет API-запросы в MadeTask, обрабатывает ответы и ошибки

MadeTask API: Внешний API для создания задач, подтверждения выполнения и выплат

MadeTask: Внешняя облачная система, проводит операции по задачам и оплатам

Бухгалтерия: Контролирует статус выплат и ошибки, но не выполняет ручные переводы

3.3 Тип интеграции

Для взаимодействия CRM WebStyle и MadeTask используется API-интеграция по HTTPS. Основной формат обмена данными — JSON. Запросы должны содержать метод, URL, заголовки авторизации и тело запроса.

Основные типы взаимодействий представлены в Таблице 3.

Таблица 3 – Типы взаимодействия

Тип взаимодействия	Где используется	Обоснование
Синхронное	Создание задачи, проверка исполнителя,	CRM должна получить быстрый ответ от MadeTask
Асинхронное отложенное	/ Получение финального статуса выплаты	Платеж может обрабатываться не мгновенно
Повтор запроса	Ошибки сети, временная недоступность API	Нужно снизить риск потери операции
Логирование	Все интеграционные вызовы	Нужно обеспечить аудит и разбор ошибок

Общие правила:

- CRM является инициатором большинства запросов.
- MadeTask возвращает результат операции и внешний идентификатор сущности.
- В CRM сохраняется связь между внутренней сущностью и внешней сущностью MadeTask.
- Все API-запросы логируются.
- При ошибке API CRM фиксирует ошибку и уведомляет ответственного пользователя.
- При временной ошибке выполняется повторная отправка запроса.
- Выплата не может быть инициирована до приемки задачи менеджером.
- Если исполнитель не зарегистрирован или не привязал счет, задача не может быть передана на автоматическую выплату.

3.4 Основные интеграционные сценарии

В рамках проекта выделяются следующие сценарии взаимодействия CRM и MadeTask:

- Создание задачи в MadeTask при создании задачи в CRM.
- Регистрация исполнителя / проверка готовности исполнителя к выплатам.
- Передача статуса «Готово» после выполнения задачи.
- Подтверждение выполнения после приемки менеджером.
- Инициация и фиксация выплаты.
- Обработка ошибок интеграции.

Диаграммы последовательностей по основным сценариям приведены в приложениях:

Приложение Д — UML Sequence: создание задачи и проверка исполнителя

Приложение Е — UML Sequence: выполнение, приемка и выплата

Приложение Ж — UML Sequence: обработка ошибок интеграции

3.5 Требования к интеграционному обмену

Требования представлены в Таблице 4.

Таблица 4 – Требования к интеграционному обмену

№	Требование
INT-01	CRM должна отправлять запросы к MadeTask по HTTPS.
INT-02	Формат обмена данными — JSON.
INT-03	Все запросы должны содержать авторизационный заголовок.
INT-04	CRM должна сохранять внешний ID задачи MadeTask.
INT-05	CRM должна сохранять внешний ID платежа MadeTask.
INT-06	Все успешные и ошибочные запросы должны логироваться.
INT-07	При временной ошибке должен выполняться повтор запроса.
INT-08	При ошибке авторизации должен уведомляться администратор.
INT-09	При ошибке выплаты должен уведомляться менеджер и бухгалтерия.
INT-10	Выплата не должна запускаться до приемки задачи менеджером.
INT-11	Если исполнитель не готов к выплатам, CRM должна остановить процесс и отправить уведомление.
INT-12	Система должна исключать повторную оплату одной и той же задачи.

Выводы

В рамках проектирования интеграции определены участники обмена, тип интеграции, основные сценарии взаимодействия и исключительные ситуации. Основной интеграционный контур строится вокруг CRM WebStyle, которая управляет жизненным циклом задачи и обращается к MadeTask через API для создания задач, подтверждения выполнения и проведения выплат.

Диаграммы последовательностей показывают не только основной успешный сценарий, но и критически важные альтернативные ветки: отсутствие регистрации исполнителя, отсутствие счета, возврат задачи на доработку, ошибки API, конфликт статусов и повторную отправку запросов.

Таким образом, блок интеграции формирует техническую основу для следующего раздела — детального описания методов MadeTask API, параметров запросов, форматов ответов и ограничений обмена.

4 РАБОТА С API MADETASK

4.1 Назначение API MadeTask в проекте

API MadeTask используется для интеграции CRM WebStyle с внешним облачным сервисом MadeTask. Основная задача API — обеспечить автоматический обмен данными между CRM и MadeTask без необходимости работы пользователей в интерфейсе MadeTask.

В рамках проекта API используется для следующих целей:

- создания и учета исполнителей;
- проверки готовности исполнителя к выплатам;
- создания задачи во внешнем сервисе;
- передачи статусов выполнения;
- подтверждения приемки работы;
- инициирования выплаты;
- получения статуса платежной операции.

Согласно официальной документации MadeTask Customer API, API включает методы для создания исполнителя, управления задачами и запуска выплат на платежные инструменты исполнителей.

4.2 Общие принципы взаимодействия

Интеграция строится по принципу:

CRM WebStyle — мастер-источник операционных данных. Это означает, что CRM хранит:

- проекты;
- задачи;
- пользователей;
- роли;
- результаты работ;
- бизнес-статусы задач;
- историю приемки;
- связь с внешними объектами MadeTask.

MadeTask хранит:

- данные исполнителя, необходимые для выплат;
- платежные инструменты;
- внешние задачи;
- статусы выплат;
- платежные операции.

Для исключения дублирования данных CRM должна хранить только:

- contractor_id MadeTask;
- made_task_id;
- payout_id;
- статус синхронизации;
- статус выплаты;
- технические атрибуты интеграции.

4.3 Базовые технические условия обмена

Тип интеграции API-интеграция

Протокол HTTPS

Формат данных JSON

Авторизация API-токен / Bearer token

Основной контур CRM → MadeTask API

Базовый URL <https://api.customer.madetask.ru/v1>

Версия API v1, при необходимости v2

Идемпотентность Обязательна для создания задач и выплат

Логирование Все запросы и ответы фиксируются в журнале интеграции

4.4 Методы API и данные

Методы API необходимые для проекта представлены в таблице 5.

Таблица 5 – Методы API

Операция	Метод MadeTask	Назначение	Когда вызывается
Создать/зарегистрировать исполнителя	Create Contractor	Создание карточки исполнителя или получение идентификатора contractor_id для дальнейших задач и выплат.	При подключении нового подрядчика или первичной синхронизации.
Добавить платежные данные	Add Payment Details by Token	Привязка платежного инструмента исполнителя по токenu, если сценарий поддерживается API.	После регистрации исполнителя или при обновлении реквизитов.
Создать задачу	Create Task	Создание задачи MadeTask, связанной с задачей CRM.	После создания задачи менеджером в CRM.
Изменить/подтвердить выполнение задачи	Update/Complete Task	Передача статуса готовности или выполнения.	Когда исполнитель отметил «Готово» или менеджер принял работу.
Создать выплату	Create Payout	Инициация выплаты исполнителю на привязанный платежный инструмент.	После приемки работы менеджером.
Получить статус выплаты	Get Payout / webhook	Получение результата обработки платежа.	По webhook или периодической сверке.
Получить статус исполнителя	Get Contractor	Проверка регистрации, KYC и платежных данных.	Перед назначением задачи и перед выплатой.

Таблица 6 – Данные исполнителя

Группа данных	Поля	Комментарий
Идентификация	ФИО, email, телефон, страна, налоговый статус	Используются для регистрации и идентификации исполнителя
Платежные данные	Тип платежного инструмента, токен / идентификатор платежного инструмента, валюта	CRM не должна хранить полные реквизиты карты или счета
Проверка	Статус KYC, статус верификации, дата обновления	Без успешной проверки выплата может быть недоступна
Связь с CRM	crm_user_id, contractor_id, дата подтверждения аккаунта	Нужны для связи исполнителя в двух системах
Статус готовности	not_registered, registered, payment_details_required, ready_for_payout	Используется для блокировки выплаты при неполных данных

Таблица 7 – Данные задачи

Поле	Описание	Обязательность
external_id	ID задачи в CRM	Да
contractor_id	ID исполнителя в MadeTask	Да
title	Название задачи	Да
description	Описание работ	Да
amount	Сумма выплаты	Да
currency	Валюта	Да
deadline	Срок выполнения	Желательно
project	Проект CRM	Желательно
result_url	Ссылка на результат работы	При завершении

Таблица 8 – Данные выплаты

Поле	Описание	Обязательность
external_id	ID выплаты в CRM	Да
task_id	ID задачи MadeTask	Да
contractor_id	ID исполнителя MadeTask	Да
amount	Сумма выплаты	Да
currency	Валюта выплаты	Да
description	Назначение платежа	Желательно
idempotency_key	Ключ идемпотентности	Да

4.5 Примеры запросов

Ниже представлены аналитические примеры различных запросов. Финальный состав полей необходимо сверить с актуальной спецификацией MadeTask.

Создание исполнителя

POST https://api.customer.madetask.ru/v1/contractors
Authorization: Bearer <api_token>
Content-Type: application/json
Idempotency-Key: <crm_contractor_uuid>

```
{
  "external_id": "CRM-CONTRACTOR-125",
  "first_name": "Иван",
  "last_name": "Петров",
  "email": "ivan.petrov@example.com",
  "phone": "+79991234567",
  "country": "RU",
  "tax_status": "self_employed"
}
```

Успешный ответ

201 Created

Content-Type: application/json

```
{
  "contractor_id": "mt_contractor_987",
  "external_id": "CRM-CONTRACTOR-125",
  "status": "registered",
  "payment_status": "payment_details_required"
}
```

Проверка статуса исполнителя

GET https://api.customer.madetask.ru/v1/contractors/mt_contractor_987

Authorization: Bearer <api_token>

Content-Type: application/json

Успешный ответ

```
{
  "contractor_id": "mt_contractor_987",
  "status": "verified",
  "kyc_status": "approved",
  "payment_status": "ready_for_payout",
  "updated_at": "2026-05-14T12:00:00+03:00"
}
```

Создание задачи

POST https://api.customer

POST https://api.customer.madetask.ru/v1/tasks

Authorization: Bearer <api_token>

Idempotency-Key: <crm_task_uuid>

Content-Type: application/json

```
{
  "external_id": "CRM-TASK-10245",
  "contractor_id": "mt_contractor_987",
  "title": "Дизайн главной страницы сайта",
  "description": "Подготовить desktop/mobile макет главной страницы по
техническому заданию",
  "amount": 25000,
  "currency": "RUB",
  "deadline": "2026-05-15",
  "project": "WebStyle project #74"
}
```

Успешный ответ

201 Created

Content-Type: application/json

```
{
  "task_id": "mt_task_123",
}
```

```

    "external_id": "CRM-TASK-10245",
    "status": "created",
    "contractor_id": "mt_contractor_987",
    "amount": 25000,
    "currency": "RUB"
  }
  Ошибка создания задачи
  400 Bad Request
  Content-Type: application/json
  {
    "error": "validation_error",
    "message": "contractor_id is required",
    "fields": {
      "contractor_id": "Required field"
    }
  }
}

```

Изменение статуса задачи

Запрос выполняется, когда исполнитель отмечает задачу как «Готово» в CRM.

```

PATCH https://api.customer.madetask.ru/v1/tasks/mt_task_123
Authorization: Bearer <api_token>
Content-Type: application/json
{
  "status": "completed",
  "completed_at": "2026-05-14T16:30:00+03:00",
  "result_url": "https://crm.webstyle.local/tasks/10245/result"
}
Успешный ответ
{
  "task_id": "mt_task_123",
  "status": "completed",
  "updated_at": "2026-05-14T16:30:05+03:00"
}

```

Подтверждение выполнения задачи

Запрос выполняется после приемки работы менеджером.

```

POST https://api.customer.madetask.ru/v1/tasks/mt_task_123/complete
Authorization: Bearer <api_token>
Content-Type: application/json
{
  "accepted_by": "manager_45",
  "accepted_at": "2026-05-14T17:00:00+03:00",

```

```
"comment": "Работа принята без замечаний"
}
Успешный ответ
{
  "task_id": "mt_task_123",
  "status": "confirmed",
  "accepted_at": "2026-05-14T17:00:00+03:00"
}
```

Создание выплаты

Запрос выполняется после того, как задача принята менеджером.

```
POST https://api.customer.madetask.ru/v1/payouts
Authorization: Bearer <api_token>
Idempotency-Key: <crm_task_uuid>:payout
Content-Type: application/json
{
  "external_id": "CRM-PAYOUT-10245",
  "task_id": "mt_task_123",
  "contractor_id": "mt_contractor_987",
  "amount": 25000,
  "currency": "RUB",
  "description": "Оплата задачи CRM-TASK-10245"
}
Успешный ответ
202 Accepted
Content-Type: application/json
{
  "payout_id": "mt_payout_555",
  "external_id": "CRM-PAYOUT-10245",
  "status": "processing",
  "amount": 25000,
  "currency": "RUB"
}
```

Получение статуса выплаты

```
GET https://api.customer.madetask.ru/v1/payouts/mt_payout_555
Authorization: Bearer <api_token>
Content-Type: application/json
Успешный ответ
{
  "payout_id": "mt_payout_555",
  "status": "paid",
  "paid_at": "2026-05-14T17:05:00+03:00",
  "amount": 25000,
  "currency": "RUB"
}
```

Ответ при ошибке выплаты

```
{
  "payout_id": "mt_payout_555",
  "status": "failed",
  "error_code": "payment_details_missing",
  "message": "Payment details are missing or invalid"
}
```

Коды ответов и обработка ошибок представлены в таблице 9.

Таблица 9 – Коды ошибок

Код	Значение	Действие CRM
200 OK	Запрос выполнен успешно	Обновить статус сущности
201 Created	Объект создан	Сохранить внешний ID
202 Accepted	Операция принята в обработку	Сохранить ID операции и ожидать финальный статус
400 Bad Request	Ошибка валидации	Показать ошибку пользователю, не повторять автоматически
401 Unauthorized	Ошибка авторизации	Уведомить администратора, остановить интеграцию
403 Forbidden	Недостаточно прав	Уведомить администратора
404 Not Found	Объект не найден	Запустить сверку по внешнему ID
409 Conflict	Конфликт статусов	Получить актуальный статус из MadeTask
429 Too Many Requests	Превышен лимит запросов	Повторить позже с задержкой
500 Internal Server Error	Ошибка MadeTask	Повторить запрос, затем уведомить администратора
Timeout	Нет ответа	Повторить запрос, затем зафиксировать ошибку

4.6 Ограничения и требования к безопасности

Таблица 10 – Ограничения

Ограничение / условие	Как учесть в проекте
Облачная модель MadeTask	Все операции по выплатам проходят через MadeTask; CRM хранит только идентификаторы и статусы
Регистрация исполнителей	Исполнитель должен самостоятельно зарегистрироваться по ссылке
Верификация исполнителей	До первой выплаты CRM должна проверить статус KYC и платежного инструмента
Платежные реквизиты	CRM не хранит полные реквизиты карт или счетов
Лимиты выплат	Проверять сумму выплаты перед отправкой
Лимиты запросов API	При 429 использовать очередь и повтор с задержкой
Идемпотентность	Для создания задач и выплат использовать уникальный ключ операции
Безопасность токенов	Хранить API-токены в защищенном хранилище
Аудит	Логировать запрос, ответ, время, внешний ID, код ошибки
Повторные выплаты	Запрещать создание выплаты, если по задаче уже есть payout_id

Таблица 11 – Требования к безопасности

№	Требование
API-SEC-01	Все запросы должны выполняться только по HTTPS.
API-SEC-02	API-токен должен храниться в защищенном хранилище.
API-SEC-03	Токены не должны отображаться в интерфейсе пользователя и логах.
API-SEC-04	Полные платежные реквизиты исполнителей не должны храниться в CRM.
API-SEC-05	Доступ к журналу интеграции должен быть ограничен администраторами.
API-SEC-06	В логах запрещено хранить чувствительные платежные данные.
API-SEC-07	Все операции создания выплат должны быть идемпотентными.

Выводы

Для реализации интеграции CRM WebStyle с MadeTask необходимо использовать Customer API MadeTask. Основными методами являются создание и проверка исполнителя, создание задачи, изменение статуса задачи, подтверждение выполнения, создание выплаты и получение статуса выплаты.

CRM должна выступать мастер-системой для операционных данных, а MadeTask — внешним сервисом учета исполнителей и проведения выплат. Для обеспечения надежности интеграции необходимо использовать HTTPS, JSON, авторизацию по API-токену, идемпотентные ключи, журналирование операций и обработку ошибок API.

Описанные методы, структуры запросов и ограничения формируют техническую основу для разработки интеграционного слоя и дальнейшего планирования задач реализации.

5 ДОКУМЕНТИРОВАНИЕ

5.1 Назначение раздела

Настоящий раздел фиксирует требования к системе и план разработки интеграции CRM WebStyle с сервисом MadeTask. Документ предназначен для использования командами разработки, тестирования и сопровождения как основа для реализации, оценки трудозатрат и приемки результата.

5.2 Границы решения

В рамках реализации:

Входит в объем:

- доработка CRM (задачи, статусы, роли);
- интеграционный модуль (вызовы API MadeTask);
- автоматизация выплат;
- журнал интеграции и обработка ошибок;
- отчетность по выплатам.

Не входит в объем:

- разработка функционала MadeTask;
- хранение платежных реквизитов в CRM;
- банковские операции вне MadeTask.

5.3 Функциональные требования (FR)

Функциональные требования представлены в таблице 11.

Таблица 11 – Функциональные требования

Управление задачами

ID	Требование	Описание	Критерий приемки
FR-01	Создание задачи	Менеджер создает задачу с указанием исполнителя, суммы, срока	Задача создается в CRM и получает статус «Создана»
FR-02	Назначение исполнителя	Задача должна быть привязана к одному исполнителю	Исполнитель указан и доступен в карточке задачи

ID	Требование	Описание	Критерий приемки
FR-03	Синхронизация с MadeTask	При создании задачи CRM должна создать задачу в MadeTask	В CRM сохраняется made_task_id
FR-04	Обновление статусов	Изменение статуса задачи должно синхронизироваться с MadeTask	Статусы совпадают в обеих системах
FR-05	Приемка задачи	Менеджер может принять или отклонить результат	При принятии статус → «Принята»

Работа с исполнителями

ID	Требование	Описание	Критерий приемки
FR-06	Проверка исполнителя	CRM проверяет статус регистрации в MadeTask	Возвращается корректный статус
FR-07	Приглашение исполнителя	При отсутствии аккаунта отправляется ссылка	Исполнитель получает уведомление
FR-08	Связь с MadeTask	Для каждого исполнителя хранится contractor_id	Связь сохраняется в CRM
FR-09	Ограничение доступа	Исполнитель видит только свои задачи	Доступ ограничен ролью

Выполнение и приемка

ID	Требование	Описание	Критерий приемки
FR-10	Отметка «Готово»	Исполнитель может завершить задачу	Статус → «Готова к проверке»
FR-11	Передача статуса	CRM отправляет статус в MadeTask	MadeTask получает статус
FR-12	Возврат на доработку	Менеджер может вернуть задачу	Статус → «На доработке»

Выплаты

ID	Требование	Описание	Критерий приемки
FR-13	Автоматическая выплата	Выплата запускается после приемки	Платеж создается в MadeTask
FR-14	Проверка условий	Перед выплатой проверяется готовность исполнителя	Выплата не создается при ошибке
FR-15	Получение статуса выплаты	CRM получает статус платежа	Статус фиксируется в системе
FR-16	Исключение дублей	Повторная выплата невозможна	Проверяется payout_id / idempotency
FR-17	Ошибка выплаты	Ошибка фиксируется и отображается	Статус → «Ошибка выплаты»

Журнал интеграции

ID	Требование	Описание	Критерий приемки
FR-18	Логирование	Все API-запросы должны логироваться	Запись появляется в журнале
FR-19	Повтор запроса	При ошибке возможен retry	Запрос выполняется повторно
FR-20	Просмотр логов	Администратор может просматривать журнал	Доступ по роли

5.4 Нефункциональные требования (NFR)

Нефункциональные требования представлены в таблице 12.

Таблица 12 – Нефункциональные требования

Производительность

ID	Требование	Значение
NFR-01	Время ответа API	≤ 2 секунды
NFR-02	Обработка задач	≥ 1000 задач/сутки
NFR-03	Параллельные операции	≥ 100 одновременных запросов

Надежность

ID	Требование	Описание
NFR-04	Повтор запросов	Retry при ошибках сети
NFR-05	Идемпотентность	Исключение дублирования операций
NFR-06	Логирование	Все операции фиксируются

Безопасность

ID	Требование	Описание
NFR-07	HTTPS	Все запросы только по HTTPS
NFR-08	API-токены	Хранение в защищенном виде
NFR-09	Данные	Не хранить платежные реквизиты
NFR-10	Доступ	Ограничение по ролям

Масштабируемость

ID	Требование	Описание
NFR-11	Рост пользователей	Поддержка увеличения подрядчиков
NFR-12	Расширение API	Возможность добавления новых методов

5.5 Список задач разработки (Backlog)

Список задач представлены в таблице 13.

Таблица 13 – Список задач

Интеграция

Название задачи	Описание
Интеграция с MadeTask API	Реализовать базовый клиент API
Создание задачи	Реализовать POST /tasks
Обновление статуса	Реализовать PATCH /tasks
Подтверждение выполнения	Реализовать complete task
Выплаты	Реализовать POST /payouts
Получение статуса	Реализовать GET payout

CRM доработки

Название задачи	Описание
Добавление статусов	Реализовать статусы задач
Карточка исполнителя	Добавить поля MadeTask
UI для менеджера	Интерфейс управления задачами
UI для исполнителя	Интерфейс выполнения задач

Логирование и ошибки

Название задачи	Описание
Журнал интеграции	Таблица логов API
Retry-механизм	Повтор запросов
Уведомления	Ошибки и события

Тестирование

Название задачи	Описание
Unit-тесты	Проверка логики
Интеграционные тесты	Проверка API
Нагрузочные тесты	Проверка производительности

5.6 План реализации

Этап	Содержание	Срок
Этап 1	Анализ и проектирование	1 неделя
Этап 2	Разработка интеграции	2 недели
Этап 3	Разработка CRM	2 недели

Этап	Содержание	Срок
Этап 4	Тестирование	1 неделя
Этап 5	Внедрение	1 неделя

5.7 Критерии приемки

Система считается реализованной, если:

- все задачи создаются и синхронизируются с MadeTask;
- исполнитель может выполнить и сдать задачу;
- менеджер может принять задачу;
- выплата происходит автоматически;
- ошибки корректно обрабатываются;
- журнал интеграции фиксирует операции;
- отсутствуют дубли платежей;
- все сценарии BPMN реализованы.

Выводы

В рамках данного раздела сформирован полный перечень функциональных и нефункциональных требований, а также список задач для реализации интеграции CRM WebStyle с сервисом MadeTask.

Представленный план разработки и структура требований позволяют:

- обеспечить прозрачность реализации;
- снизить риски при разработке;
- упростить взаимодействие между аналитиками и разработчиками;
- обеспечить контроль качества на этапе тестирования и внедрения.

Таким образом, документ может использоваться как полноценное техническое задание для разработки системы.

ЗАКЛЮЧЕНИЕ

В рамках работы выполнен системный анализ текущего процесса взаимодействия компании WebStyle с внешними исполнителями и спроектировано целевое решение по интеграции CRM WebStyle с облачным сервисом MadeTask.

В ходе анализа установлено, что действующий процесс является частично цифровизированным, но разрозненным: задачи внешним подрядчикам передаются через мессенджеры, результаты работ фиксируются вне единого контура, а выплаты выполняются вручную через Excel и банковские сервисы. Это приводит к потере задач и правок, задержкам выплат, риску ошибок в реквизитах и отсутствию прозрачной связи между задачей, приемкой и оплатой.

В качестве целевого решения предложена интеграция CRM WebStyle с MadeTask через API. CRM остается основной рабочей системой для менеджеров, бухгалтерии и исполнителей, а MadeTask используется как внешний сервис учета исполнителей, задач и выплат. Такой подход позволяет сохранить привычный интерфейс CRM, исключить дублирование данных и автоматизировать оплату после приемки работы менеджером.

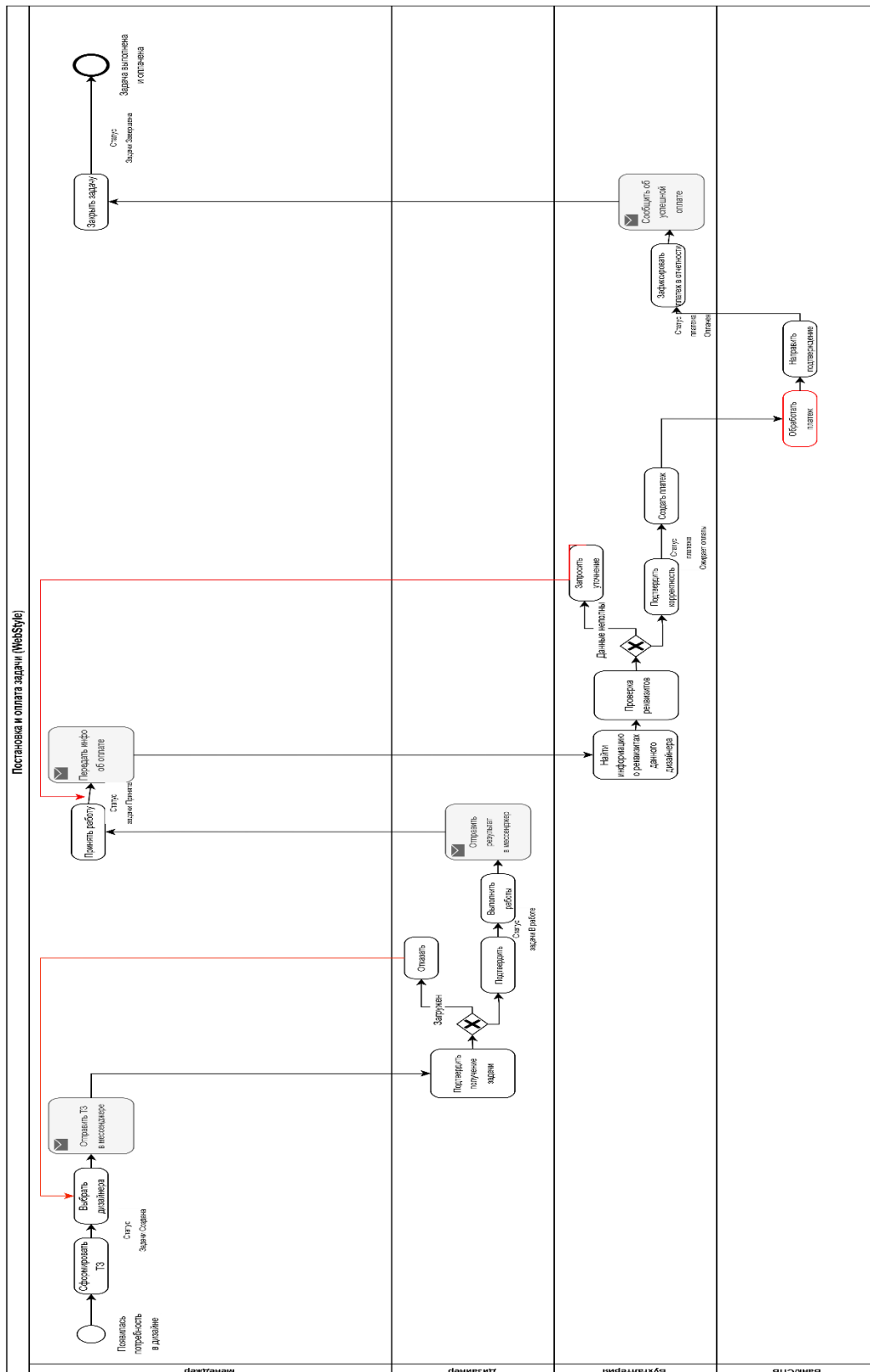
Предложенное решение обеспечивает сквозную трассируемость процесса: задача → выполнение → приемка → выплата. Это снижает зависимость от ручных операций, повышает прозрачность затрат по проектам, уменьшает нагрузку на бухгалтерию и снижает риск ошибочных или повторных выплат.

Таким образом, подготовленное техническое задание может использоваться как основа для разработки, тестирования и внедрения интеграции CRM WebStyle с MadeTask.

ПРИЛОЖЕНИЯ

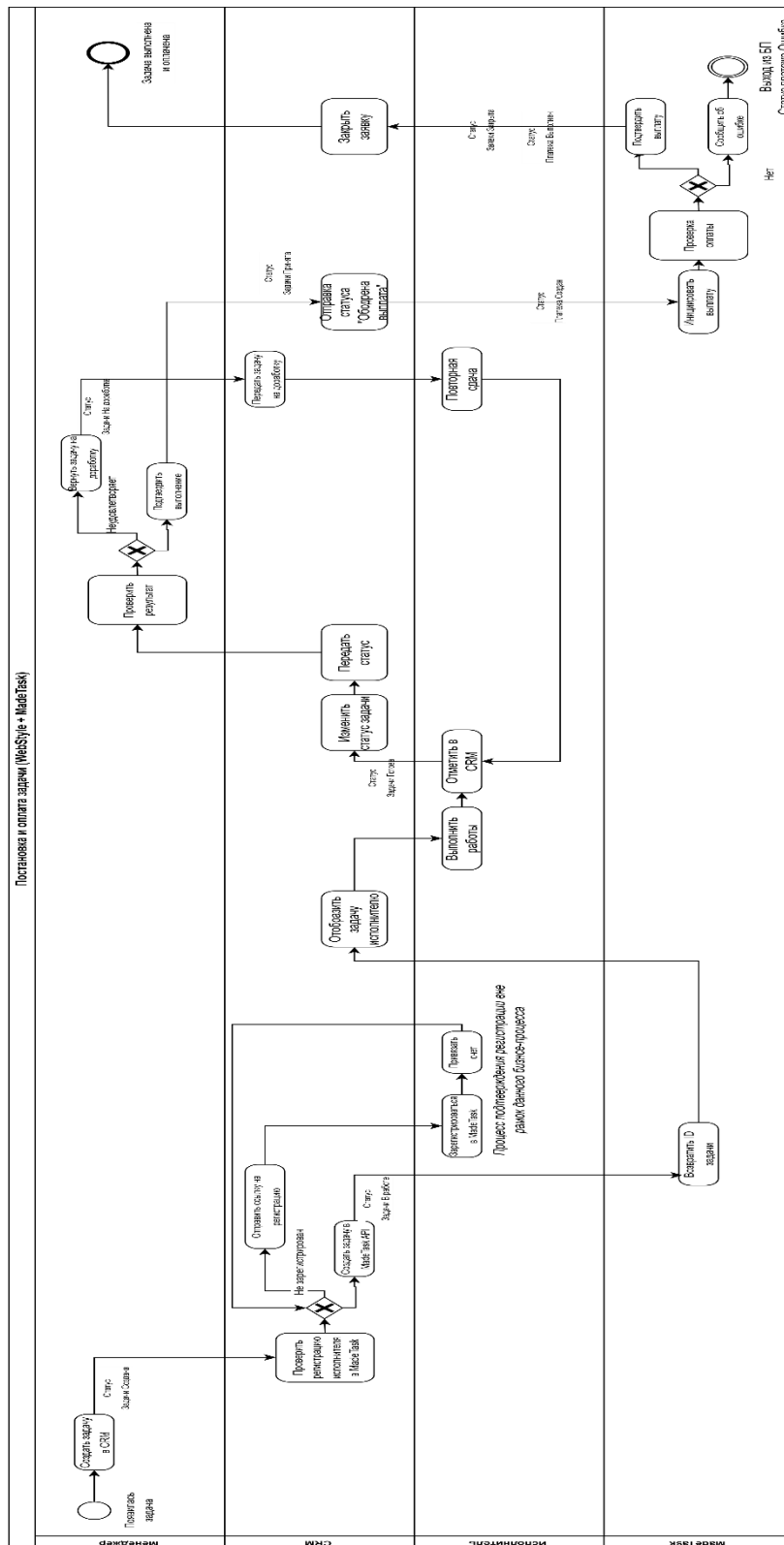
Приложение А – Процесс AS IS (WebStyle)

URL: https://drive.google.com/file/d/1x1BplWBpSP_SoV9vHnFeKT_yPjGDcsu/view?usp=drive_link
(дата обращения 27.04.2026)



Приложение Б – Процесс TO BE (WebStyle)

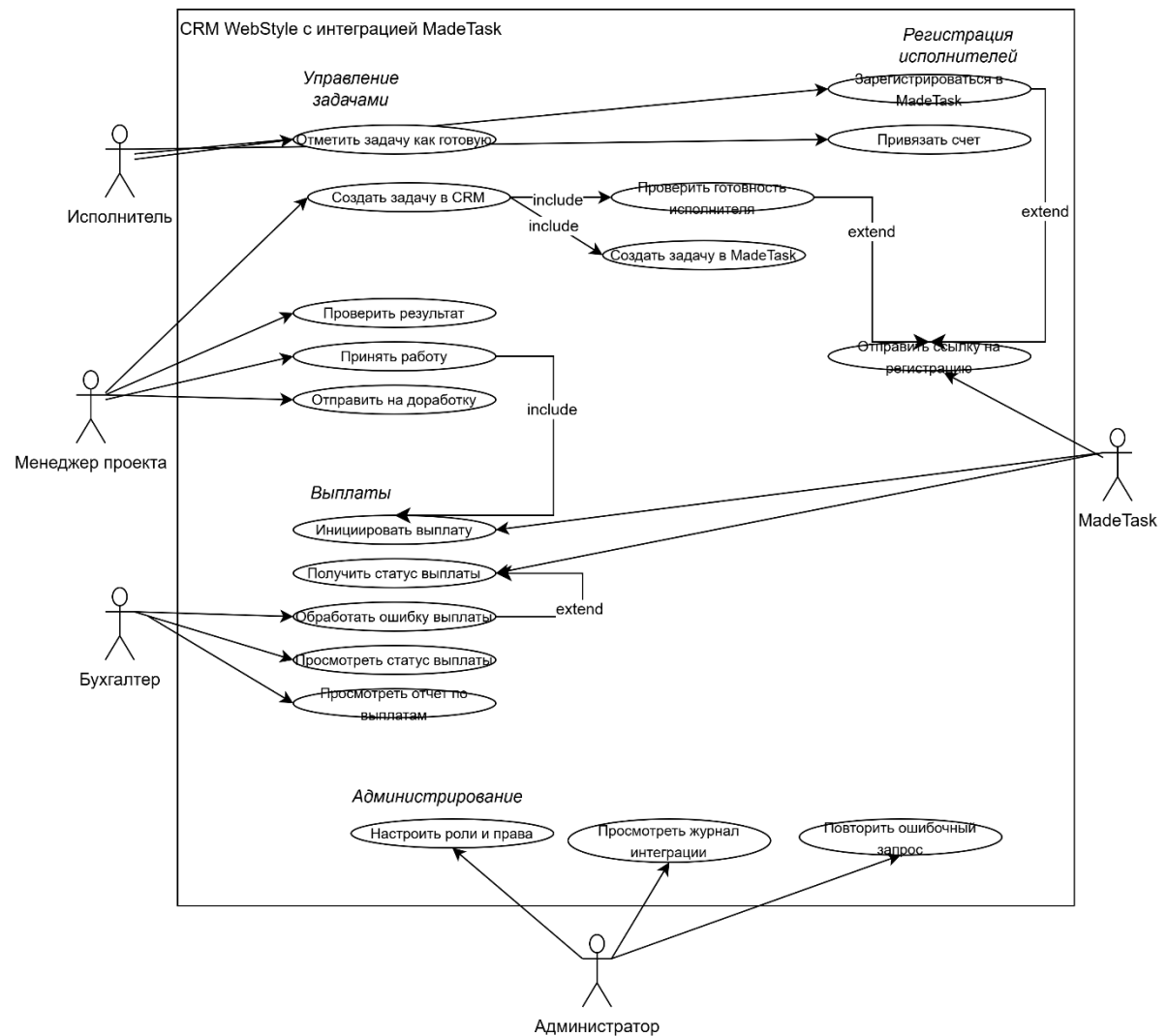
URL: https://drive.google.com/file/d/1vfp5QkalqoDGdSaGPczAVIv0kvQqkC_1/view?usp=drive_link
(дата обращения 27.04.2026)



Приложение В – Use-Case диаграмма

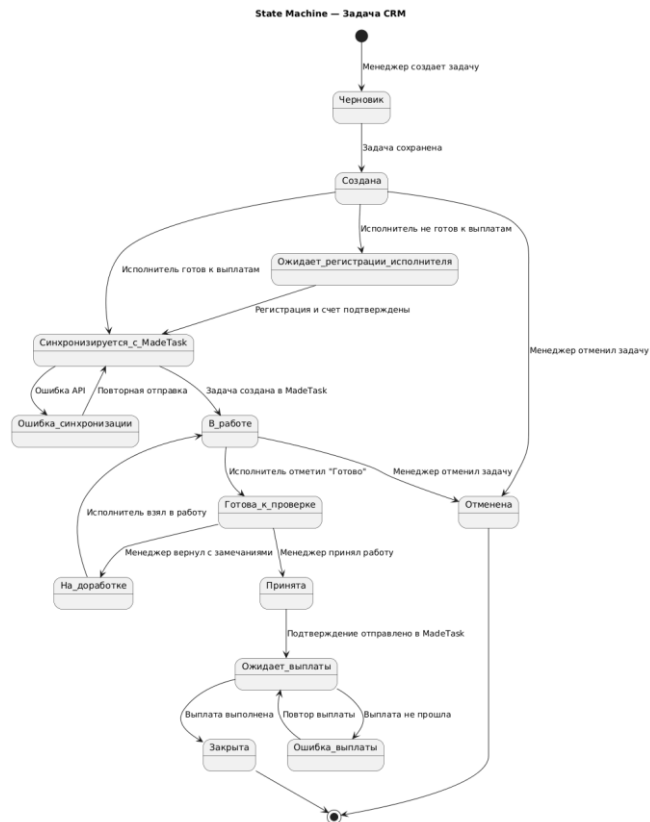
URL: https://drive.google.com/file/d/1AUu53211FR9Y8l2i8rdxG0ubtndZPp1M/view?usp=drive_link

(дата обращения 27.04.2026)



Приложение Г – Бизнес-сущности

Задача CRM



Код для PlantUML:

```

@startuml
title State Machine — Задача CRM

[*] --> Черновик : Менеджер создает задачу

Черновик --> Создана : Задача сохранена
Создана --> Ожидает_регистрации_исполнителя : Исполнитель не готов к выплатам
Создана --> Синхронизируется_c_MadeTask : Исполнитель готов к выплатам

Ожидает_регистрации_исполнителя --> Синхронизируется_c_MadeTask : Регистрация и счет подтверждены

Синхронизируется_c_MadeTask --> В_работе : Задача создана в MadeTask
Синхронизируется_c_MadeTask --> Ошибка_синхронизации : Ошибка API
Ошибка_синхронизации --> Синхронизируется_c_MadeTask : Повторная отправка

В_работе --> Готова_к_проверке : Исполнитель отметил "Готово"
В_работе --> Отменена : Менеджер отменил задачу

Готова_к_проверке --> На_доработке : Менеджер вернул с замечаниями
На_доработке --> В_работе : Исполнитель взял в работу

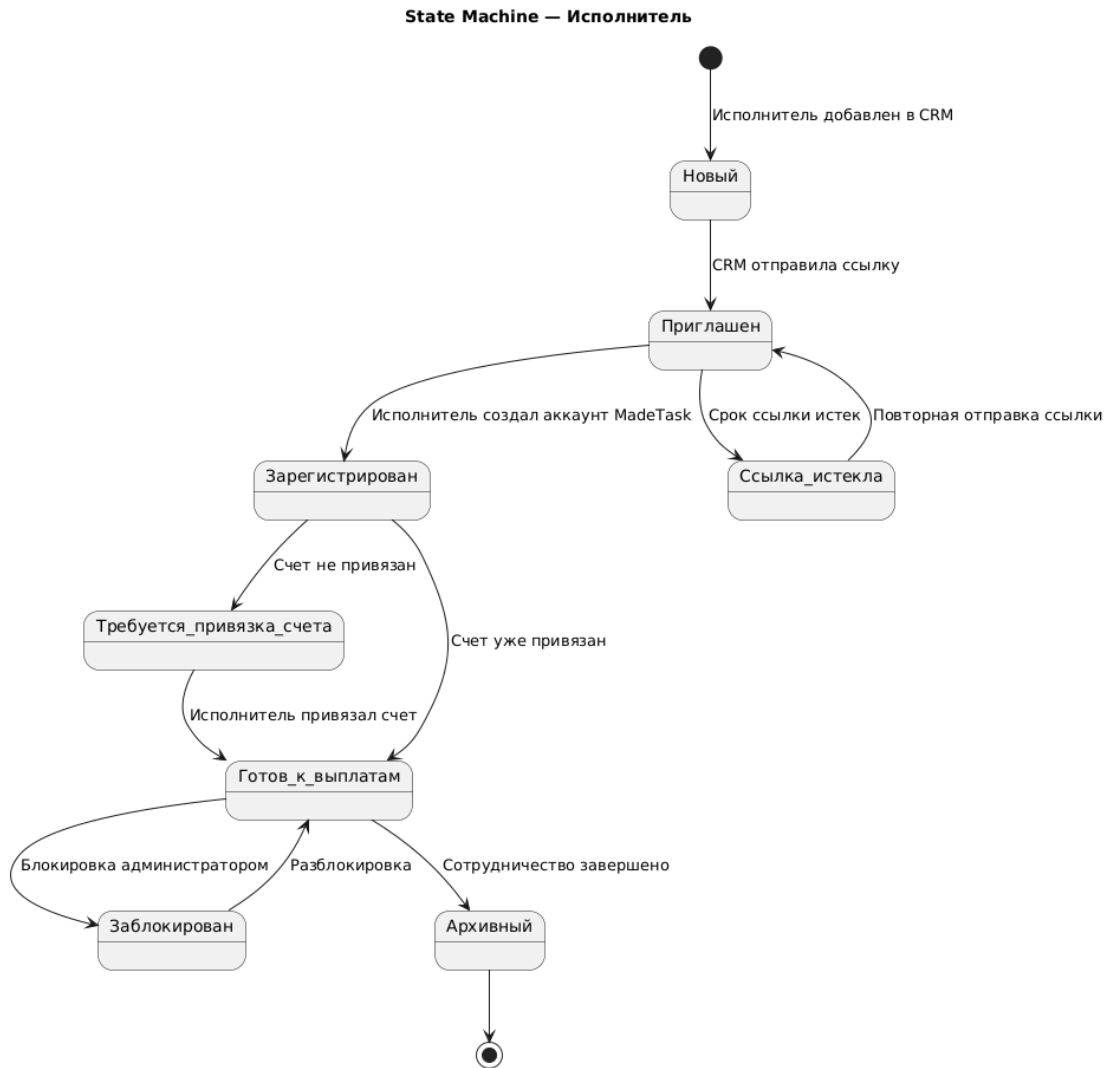
Готова_к_проверке --> Принята : Менеджер принял работу
Принята --> Ожидает_выплаты : Подтверждение отправлено в MadeTask

Ожидает_выплаты --> Ошибка_выплаты : Выплата не прошла
Ошибка_выплаты --> Ожидает_выплаты : Повтор выплаты
Ожидает_выплаты --> Закрыта : Выплата выполнена
Закрыта --> [*]

Создана --> Отменена : Менеджер отменил задачу
В_работе --> Отменена : Менеджер отменил задачу
Ожидает_регистрации_исполнителя --> Отменена : Менеджер отменил задачу
    
```

@enduml

Исполнитель



Код для PlantUML:

```

@startuml
title State Machine — Исполнитель

[*] --> Новый : Исполнитель добавлен в CRM

Новый --> Приглашен : CRM отправила ссылку
Приглашен --> Зарегистрирован : Исполнитель создал аккаунт MadeTask
Приглашен --> Ссылка_истекла : Срок ссылки истек

Ссылка_истекла --> Приглашен : Повторная отправка ссылки

Зарегистрирован --> Требуется_привязка_счета : Счет не привязан
Зарегистрирован --> Готов_к_выплатам : Счет уже привязан

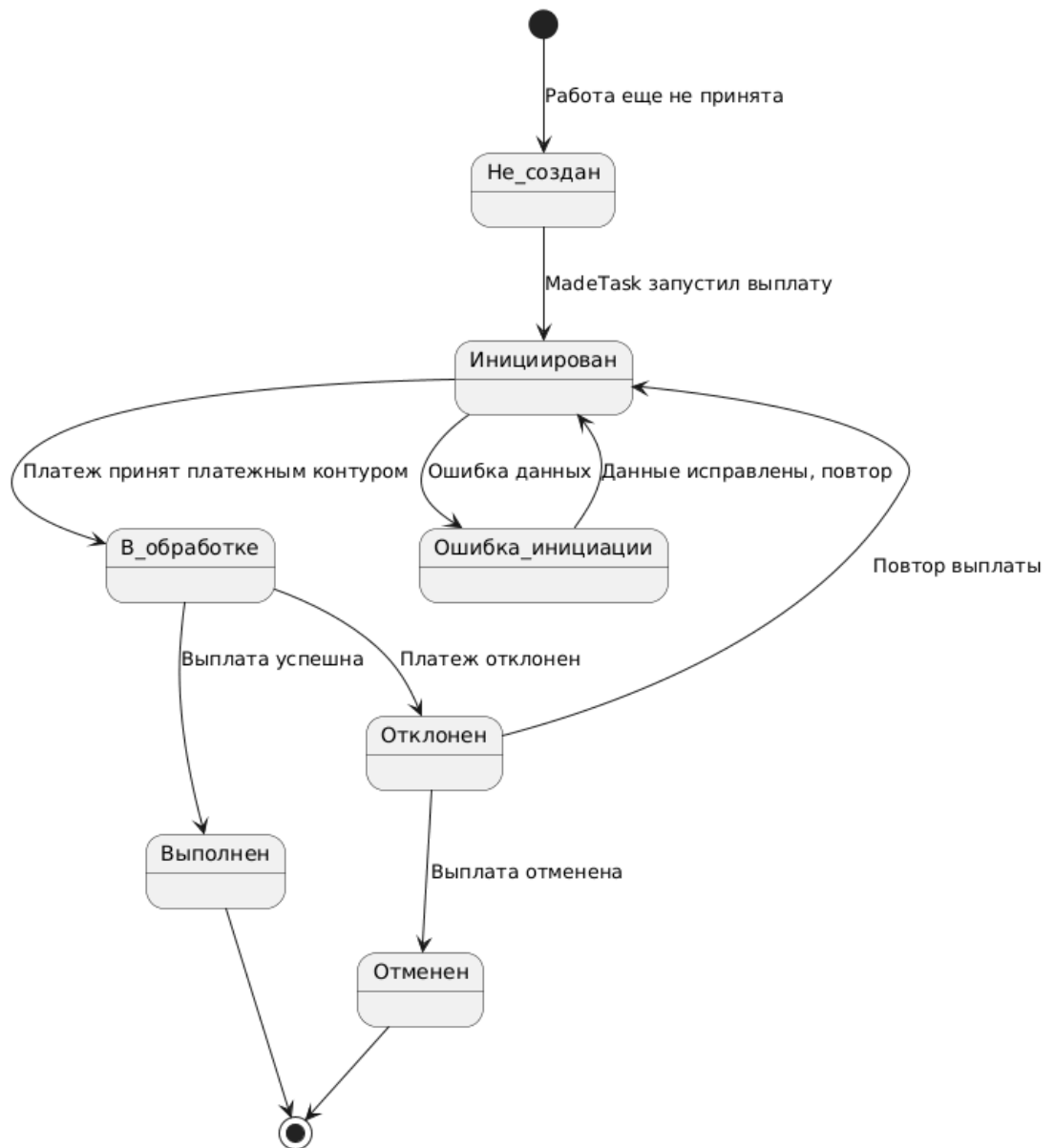
Требуется_привязка_счета --> Готов_к_выплатам : Исполнитель привязал счет

Готов_к_выплатам --> Заблокирован : Блокировка администратором
Заблокирован --> Готов_к_выплатам : Разблокировка

Готов_к_выплатам --> Архивный : Сотрудничество завершено
Архивный --> [*]

@enduml
  
```

State Machine — Платеж



Код для PlantUML:

@startuml

title State Machine — Платеж

[*] --> Не_создан : Работа еще не принята

Не_создан --> Инициирован : MadeTask запустил выплату

Инициирован --> В_обработке : Платеж принят платежным контуром

Инициирован --> Ошибка_инициации : Ошибка данных

Ошибка_инициации --> Инициирован : Данные исправлены, повтор

В_обработке --> Выполнен : Выплата успешна

В_обработке --> Отклонен : Платеж отклонен

Отклонен --> Инициирован : Повтор выплаты

Отклонен --> Отменен : Выплата отменена

Выполнен --> [*]

Отменен --> [*]

@enduml

Приложение Д – UML Sequence: создание задачи и проверка исполнителя

Сценарий начинается с того, что менеджер создает задачу в CRM. CRM проверяет, готов ли исполнитель к получению выплат через MadeTask. Если исполнитель не зарегистрирован или не привязал счет, CRM отправляет ссылку на регистрацию. После подтверждения готовности исполнителя CRM создает задачу в MadeTask и сохраняет внешний ID задачи.

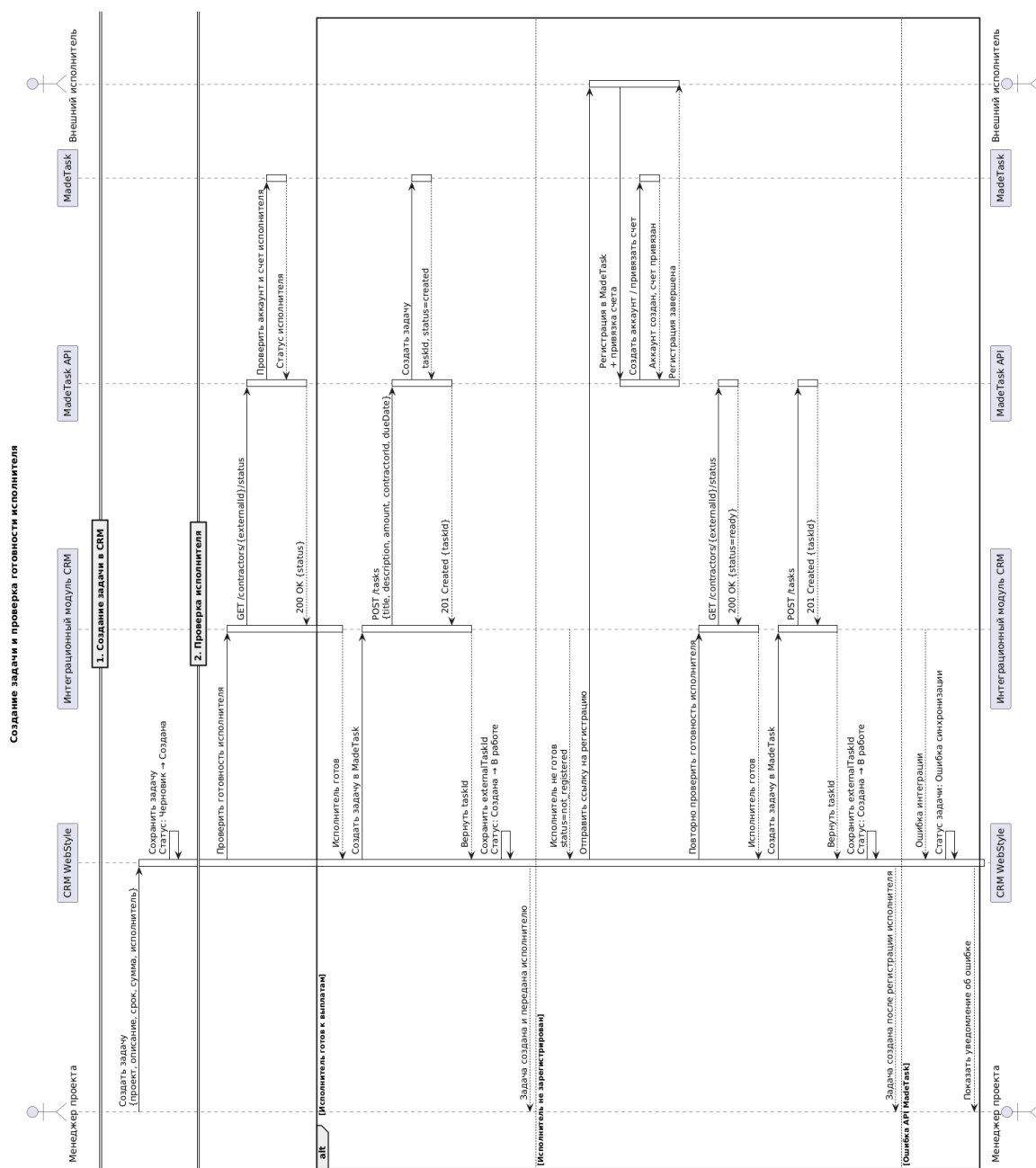
Изменение статусов

Сущность Начальный статус Конечный статус

Задача CRM Черновик В работе

Исполнитель Новый / Не готов к выплатам Готов к выплатам

Задача MadeTask Не создана Создана



Код для PlantUML:

@startuml

title Создание задачи и проверка готовности исполнителя

actor "Менеджер проекта" as Manager

participant "CRM WebStyle" as CRM

participant "Интеграционный модуль CRM" as Integration

participant "MadeTask API" as API

participant "MadeTask" as MT

actor "Внешний исполнитель" as Contractor

== 1. Создание задачи в CRM ==

Manager -> CRM : Создать задачу\n{проект, описание, срок, сумма, исполнитель}

activate CRM

CRM -> CRM : Сохранить задачу\nСтатус: Черновик → Создана

== 2. Проверка исполнителя ==

CRM -> Integration : Проверить готовность исполнителя

activate Integration

Integration -> API : GET /contractors/{externalId}/status

activate API

API -> MT : Проверить аккаунт и счет исполнителя

activate MT

MT --> API : Статус исполнителя

deactivate MT

API --> Integration : 200 OK {status}

deactivate API

alt Исполнитель готов к выплатам

Integration --> CRM : Исполнитель готов

deactivate Integration

CRM -> Integration : Создать задачу в MadeTask

activate Integration

Integration -> API : POST /tasks\n{title, description, amount, contractorId, dueDate}

activate API

API -> MT : Создать задачу

activate MT

MT --> API : taskId, status=created

deactivate MT

API --> Integration : 201 Created {taskId}

deactivate API

Integration --> CRM : Вернуть taskId

deactivate Integration

CRM -> CRM : Сохранить externalTaskId\nСтатус: Создана → В работе

CRM --> Manager : Задача создана и передана исполнителю

else Исполнитель не зарегистрирован

Integration --> CRM : Исполнитель не готов\nstatus=not_registered

deactivate Integration

```

CRM -> Contractor : Отправить ссылку на регистрацию
activate Contractor
Contractor -> API : Регистрация в MadeTask\n+ привязка счета
activate API
API -> MT : Создать аккаунт / привязать счет
activate MT
MT --> API : Аккаунт создан, счет привязан
deactivate MT
API --> Contractor : Регистрация завершена
deactivate API
deactivate Contractor

CRM -> Integration : Повторно проверить готовность исполнителя
activate Integration
Integration -> API : GET /contractors/{externalId}/status
activate API
API --> Integration : 200 OK {status=ready}
deactivate API
Integration --> CRM : Исполнитель готов
deactivate Integration

CRM -> Integration : Создать задачу в MadeTask
activate Integration
Integration -> API : POST /tasks
activate API
API --> Integration : 201 Created {taskId}
deactivate API
Integration --> CRM : Вернуть taskId
deactivate Integration

CRM -> CRM : Сохранить externalTaskId\nСтатус: Создана → В работе
CRM --> Manager : Задача создана после регистрации исполнителя

else Ошибка API MadeTask
Integration --> CRM : Ошибка интеграции
deactivate Integration
CRM -> CRM : Статус задачи: Ошибка синхронизации
CRM --> Manager : Показать уведомление об ошибке
end

deactivate CRM

@enduml

```

Приложение Е – UML Sequence: Обработка ошибок интеграции

Сценарий отражает исключительные ситуации, которые могут возникнуть при обмене данными между CRM и MadeTask. Ошибки могут возникать при создании задачи, передаче статуса, подтверждении выполнения или выплате. Для повышения надежности интеграции предусматриваются логирование, повтор запроса и уведомление ответственных пользователей.

Основные ошибки

Ошибка Причина Реакция системы

400 Bad Request Некорректные данные Зафиксировать ошибку, показать пользователю

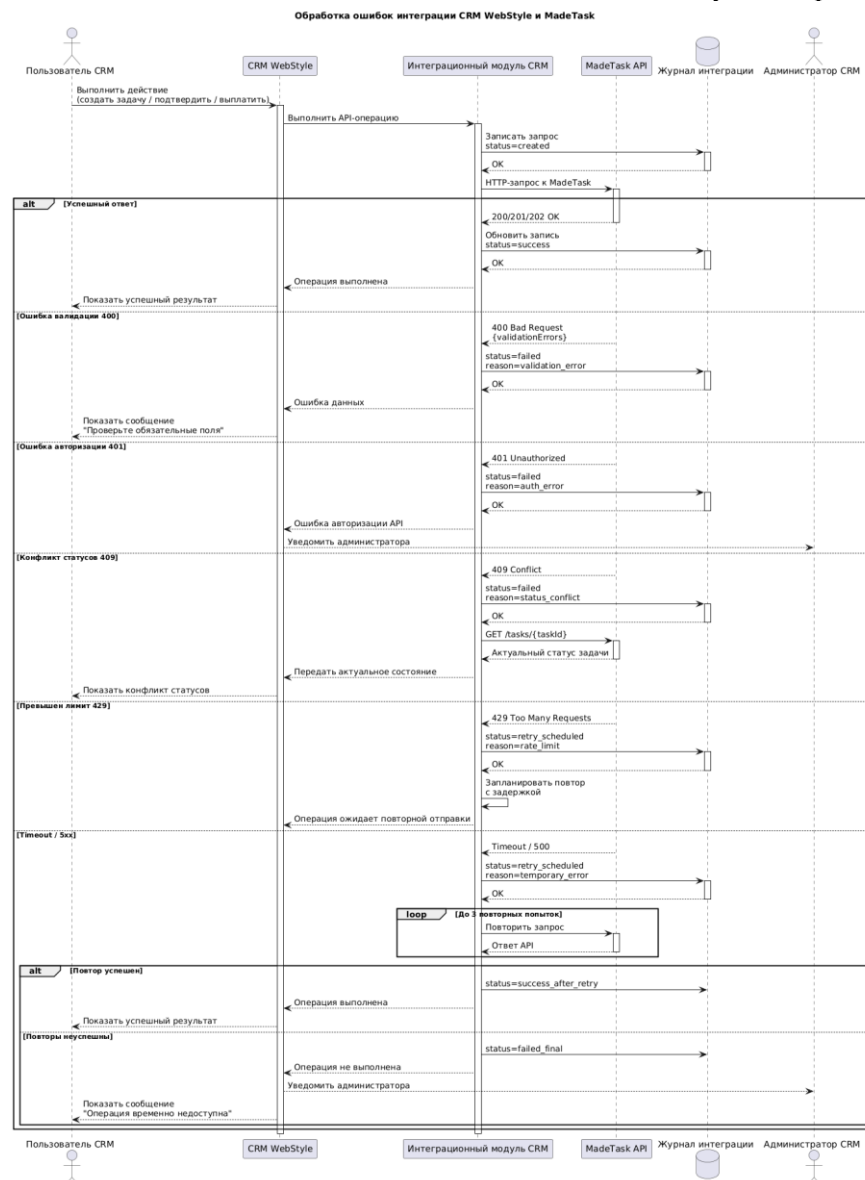
401 Unauthorized Ошибка авторизации API Уведомить администратора

404 Not Found Не найдена задача/исполнитель Заблокировать дальнейший процесс

409 Conflict Конфликт статусов Запросить актуальное состояние

429 Too Many Requests Превышен лимит запросов Повторить позже

500 / timeout Ошибка MadeTask или сети Retry и логирование



Код для PlantUML:

@startuml

title Обработка ошибок интеграции CRM WebStyle и MadeTask

actor "Пользователь CRM" as User

participant "CRM WebStyle" as CRM

participant "Интеграционный модуль CRM" as Integration

participant "MadeTask API" as API

database "Журнал интеграции" as Log

actor "Администратор CRM" as Admin

User -> CRM : Выполнить действие\n(создать задачу / подтвердить / выплатить)

activate CRM

CRM -> Integration : Выполнить API-операцию

activate Integration

Integration -> Log : Записать запрос\nstatus=created

activate Log

Log --> Integration : OK

deactivate Log

Integration -> API : HTTP-запрос к MadeTask

activate API

alt Успешный ответ

API --> Integration : 200/201/202 OK

deactivate API

Integration -> Log : Обновить запись\nstatus=success

activate Log

Log --> Integration : OK

deactivate Log

Integration --> CRM : Операция выполнена

CRM --> User : Показать успешный результат

else Ошибка валидации 400

API --> Integration : 400 Bad Request\n{validationErrors}

deactivate API

Integration -> Log : status=failed\nreason=validation_error

activate Log

Log --> Integration : OK

deactivate Log

Integration --> CRM : Ошибка данных

CRM --> User : Показать сообщение\n"Проверьте обязательные поля"

else Ошибка авторизации 401

API --> Integration : 401 Unauthorized

deactivate API

Integration -> Log : status=failed\nreason=auth_error

activate Log

Log --> Integration : OK

deactivate Log

Integration --> CRM : Ошибка авторизации API

CRM --> Admin : Уведомить администратора


```

else Конфликт статусов 409
    API --> Integration : 409 Conflict
    deactivate API
    Integration -> Log : status=failed\nreason=status_conflict
    activate Log
    Log --> Integration : OK
    deactivate Log
    Integration -> API : GET /tasks/{taskId}
    activate API
    API --> Integration : Актуальный статус задачи
    deactivate API
    Integration --> CRM : Передать актуальное состояние
    CRM --> User : Показать конфликт статусов

else Превышен лимит 429
    API --> Integration : 429 Too Many Requests
    deactivate API
    Integration -> Log : status=retry_scheduled\nreason=rate_limit
    activate Log
    Log --> Integration : OK
    deactivate Log
    Integration -> Integration : Запланировать повтор\nпс задержкой
    Integration --> CRM : Операция ожидает повторной отправки

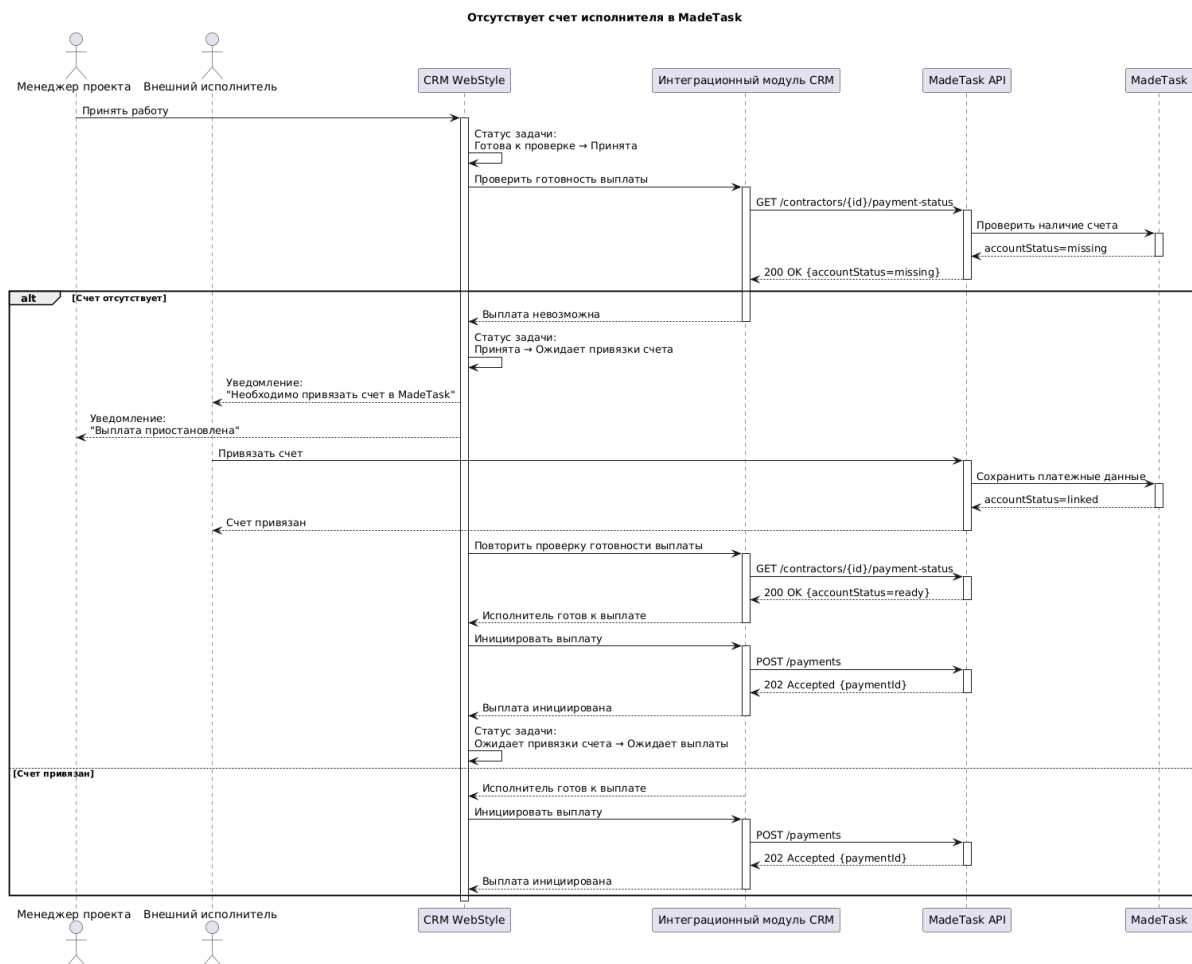
else Timeout / 5xx
    API --> Integration : Timeout / 500
    deactivate API
    Integration -> Log : status=retry_scheduled\nreason=temporary_error
    activate Log
    Log --> Integration : OK
    deactivate Log
    loop До 3 повторных попыток
        Integration -> API : Повторить запрос
        activate API
        API --> Integration : Ответ API
        deactivate API
    end
    alt Повтор успешен
        Integration -> Log : status=success_after_retry
        Integration --> CRM : Операция выполнена
        CRM --> User : Показать успешный результат
    else Повторы неуспешны
        Integration -> Log : status=failed_final
        Integration --> CRM : Операция не выполнена
        CRM --> Admin : Уведомить администратора
        CRM --> User : Показать сообщение\n"Операция временно недоступна"
    end
end
deactivate Integration
deactivate CRM

@enduml

```

Приложение Ж – UML Sequence: Сценарий отсутствия счета у исполнителя

Данный сценарий важен, потому что по условиям проекта исполнитель должен самостоятельно зарегистрироваться в MadeTask и привязать счет для оплаты. Если счет отсутствует, автоматическая выплата невозможна. CRM должна остановить процесс выплаты, уведомить исполнителя и менеджера, а после привязки счета повторить проверку.



Код для PlantUML:

@startuml

title Отсутствует счет исполнителя в MadeTask

actor "Менеджер проекта" as Manager

actor "Внешний исполнитель" as Contractor

participant "CRM WebStyle" as CRM

participant "Интеракционный модуль CRM" as Integration

participant "MadeTask API" as API

participant "MadeTask" as MT

Manager -> CRM : Принять работу

activate CRM

CRM -> CRM : Статус задачи: \nГотова к проверке → Принята

CRM -> Integration : Проверить готовность выплаты

activate Integration

Integration -> API : GET /contractors/{id}/payment-status

activate API

API -> MT : Проверить наличие счета

```

activate MT
MT --> API : accountStatus=missing
deactivate MT
API --> Integration : 200 OK {accountStatus=missing}
deactivate API

alt Счет отсутствует
    Integration --> CRM : Выплата невозможна
    deactivate Integration
    CRM -> CRM : Статус задачи:\nПринята → Ожидает привязки счета
    CRM --> Contractor : Уведомление:\n"Необходимо привязать счет в MadeTask"
    CRM --> Manager : Уведомление:\n"Выплата приостановлена"
    Contractor -> API : Привязать счет
    activate API
    API -> MT : Сохранить платежные данные
    activate MT
    MT --> API : accountStatus=linked
    deactivate MT
    API --> Contractor : Счет привязан
    deactivate API
    CRM -> Integration : Повторить проверку готовности выплаты
    activate Integration
    Integration -> API : GET /contractors/{id}/payment-status
    activate API
    API --> Integration : 200 OK {accountStatus=ready}
    deactivate API
    Integration --> CRM : Исполнитель готов к выплате
    deactivate Integration
    CRM -> Integration : Инициировать выплату
    activate Integration
    Integration -> API : POST /payments
    activate API
    API --> Integration : 202 Accepted {paymentId}
    deactivate API
    Integration --> CRM : Выплата инициирована
    deactivate Integration
    CRM -> CRM : Статус задачи:\nОжидает привязки счета → Ожидает выплаты
else Счет привязан
    Integration --> CRM : Исполнитель готов к выплате
    deactivate Integration
    CRM -> Integration : Инициировать выплату
    activate Integration
    Integration -> API : POST /payments
    activate API
    API --> Integration : 202 Accepted {paymentId}
    deactivate API
    Integration --> CRM : Выплата инициирована
    deactivate Integration
end

deactivate CRM

@enduml

```